

Bod AI — Technical Project Report

Multi-agent strategic decision simulation for financial services

AI boardroom with live debate streaming, explainable outcomes, and interactive visualization

Document type: Technical overview for academic submission

Product name: Bod AI (AI Boardroom)

API version: 0.8.0 | **Repository layout:** monorepo (backend + frontend)

1. Executive summary

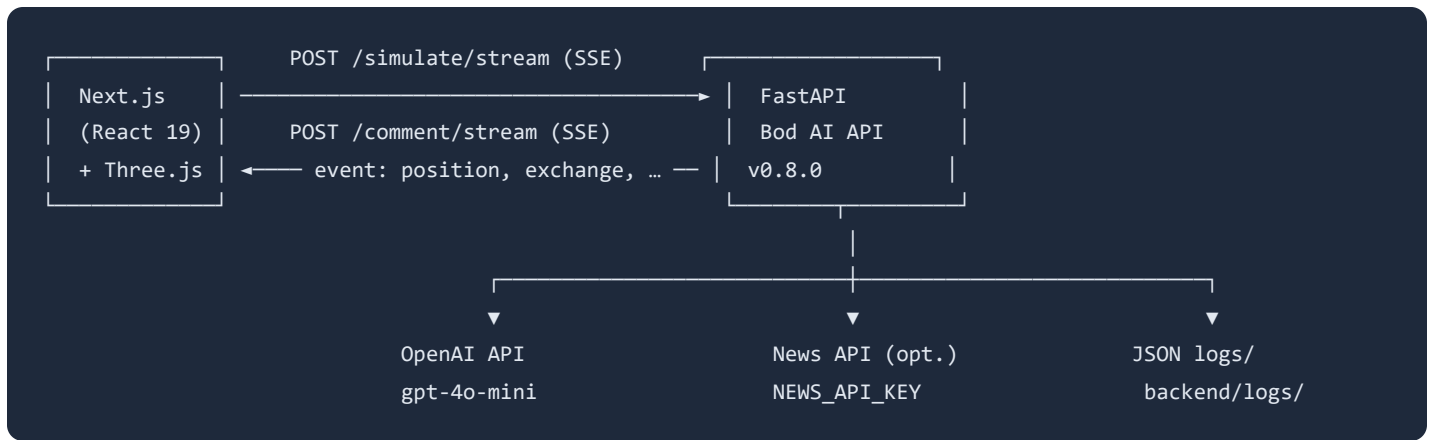
Bod AI is a full-stack application that simulates a corporate board deliberation on arbitrary business scenarios. Four LLM-backed executive personas (CEO, CFO, CMO, Chief Risk Officer) each produce an initial position, engage in configurable multi-round debate, and converge on a collective **verdict** with **confidence**, **rationale**, **supporting arguments**, and **key disagreements**. The user interface streams results in real time via **Server-Sent Events (SSE)**, visualizes agent activity in a **Three.js** voxel office, and supports optional **news context** injection, **session persistence**, **human-in-the-loop comments** (post-session board responses), memo export, replay, and session history.

2. Objectives & learning outcomes

- Demonstrate **multi-agent orchestration** with role-specific prompts and structured JSON outputs.
- Implement **streaming UX** so users observe progressive reasoning rather than a single blocking response.
- Provide **explainability**: weighted or majority aggregation plus an LLM-generated narrative summary.
- Integrate a **modern web stack** (FastAPI + Next.js) with 3D visualization and printable reporting.

3. System architecture

The system follows a **client-server** pattern. The browser runs the Next.js SPA; the Python backend calls OpenAI and optional news APIs.



3.1 Simulation pipeline (high level)

1. **News (optional):** Scenario text is used to fetch relevant headlines when `NEWS_API_KEY` is set.
2. **Phase 1 — Initial positions:** Each agent returns stance (`approve` / `conditional` / `reject`), reasoning, and key concern.
3. **Phase 2 — Debate rounds:** For each round, every agent responds to peers with a new argument and optional stance update; prior arguments are accumulated in memory.
4. **Phase 3 — Decision:** `DecisionEngine` applies weighted scoring or majority vote, then calls the LLM to synthesize the final structured summary.
5. **Persist:** Full state is saved under `backend/logs/<session_id>.json`.

3.2 Human-in-the-loop extension

After a run completes, users may submit a comment in the live discussion panel. The backend exposes `POST /comment/stream`, which streams one response per executive directed at the stakeholder, using the same debate context and the new user utterance.

4. Technology stack (versions)

Layer	Technology	Version / notes
Runtime — backend	Python	3.10+ recommended
Web framework	FastAPI	0.111.0 (<code>requirements.txt</code>)
ASGI server	Uvicorn	0.29.0
Validation / models	Pydantic	2.7.1
LLM client	OpenAI Python SDK	≥ 1.50.0; model <code>gpt-4o-mini</code>
HTTP client	httpx	≥ 0.27.0
Config	python-dotenv	1.0.1
Runtime — frontend	Node.js	18+
Framework	Next.js (App Router)	16.2.4
UI library	React / React DOM	19.2.4
Language	TypeScript	5.x
Styling	Tailwind CSS	4.x (PostCSS)
3D graphics	Three.js	0.128.0; <code>@types/three</code>
HTTP (misc.)	axios	^1.15.1
Real-time transport	SSE (fetch + ReadableStream)	Custom client in <code>lib/api.ts</code>

5. Backend design

5.1 Modules

Path	Purpose
<code>main.py</code>	FastAPI app, CORS, SSE streaming endpoints, session lifecycle
<code>agents/base_agent.py</code>	Shared OpenAI chat completion, JSON parsing, <code>get_initial_position</code> , <code>debate_response</code> , <code>respond_to_human</code>
<code>agents/*.py</code>	Role-specific system prompts (CEO, CFO, CMO, Risk)
<code>engine/debate_engine.py</code>	Non-streaming debate loop (used by <code>POST /simulate</code>)
<code>engine/decision_engine.py</code>	Weighted / majority aggregation; LLM summary (verdict, confidence, arrays)
<code>engine/news_fetcher.py</code>	Optional market news retrieval for scenario grounding
<code>memory/memory_store.py</code>	JSON file persistence and listing
<code>models/schemas.py</code>	Pydantic request/response models

5.2 REST & streaming API summary

Method	Endpoint	Description
GET	<code>/health</code>	Health + API key loaded flag
POST	<code>/simulate</code>	Full simulation (blocking JSON)
POST	<code>/simulate/stream</code>	SSE: session, status, news, position, round_start, exchange, decision, done
POST	<code>/comment/stream</code>	SSE: stakeholder follow-up; agents reply to human input
GET	<code>/logs</code>	List saved session IDs
GET	<code>/logs/{session_id}</code>	Load session JSON

5.3 Decision logic

- **Weighted mode:** Maps stances to scores (approve=1.0, conditional=0.5, reject=0.0), multiplies by per-agent weights (default **25% each**), derives confidence and verdict bands (e.g. Approved vs Conditional vs Rejected).
- **Majority mode:** Counts final stances and picks the modal category.
- **Summary:** A follow-up completion produces structured JSON: supporting arguments, disagreements, and human-readable rationale referencing named executives and vote weights.

5.4 Configuration & security notes

- `OPENAI_API_KEY` loaded from environment or `backend/.env` (must not be committed).
- Optional `NEWS_API_KEY` for news augmentation.
- CORS allows configured local/LAN origins for development.

6. Frontend design

- `app/page.tsx` : Main boardroom UI — scenario input, weight sliders (must total 100%), debate rounds, weighted vs majority mode, SSE event reducer mapping to chat messages and agent states.
- `components/Office3D.tsx` : Three.js scene; agents highlight when “speaking.”
- `components/ConfidenceMeter.tsx` : Live weighted stance visualization.
- `components/ExportPDF.tsx` : Printable / downloadable HTML memo with scenario, verdict, rationale, weights used, transcript summary.
- `lib/api.ts` : Typed SSE client for `/simulate/stream` and `/comment/stream`.
- **Additional UX**: History sidebar, replay timeline, compare view, stance-change toasts.

7. Data & persistence

Each completed simulation is stored as a JSON document in `backend/logs/` containing scenario, initial positions, debate rounds, final decision metadata, and timestamps. This supports auditing, reloading sessions in the UI, and offline review.

8. Testing & operations

- Backend: `uvicorn main:app --reload --port 8000` ; interactive docs at `/docs`.
- Frontend: `npm run dev` (default port 3000).
- Typical run latency: on the order of tens of seconds depending on rounds, model, and network.

9. Limitations & future work

- Simulation quality depends on the LLM and prompt design; outputs are **not** financial or legal advice.
- News retrieval is optional and API-dependent.
- Possible extensions: authentication, multi-tenant storage, richer evaluation metrics, agent tool use (e.g. spreadsheets), formal test suites, CI/CD.

10. Conclusion

Bod AI demonstrates a complete **multi-agent deliberation system** with **streaming UX**, **structured aggregation**, and **explainable summaries**, implemented with industry-standard web and AI tooling suitable for coursework demonstration and further research.

Generated report — Bod AI project. Open this file in a browser and print to PDF for submission.